

Totem EV Inteligente — GoodWe Challenge

Sprint 3 — Prototipagem Funcional e Integração

Sistema de recarga para veículos elétricos integrado com conceitos de Smart Grid, com balanceamento de carga, cobrança automática e persistência de telemetria, simulando o comportamento da linha GoodWe HCA G2.

Equipe 3 — GoodWe Challenge (Turma 1CCPZ)

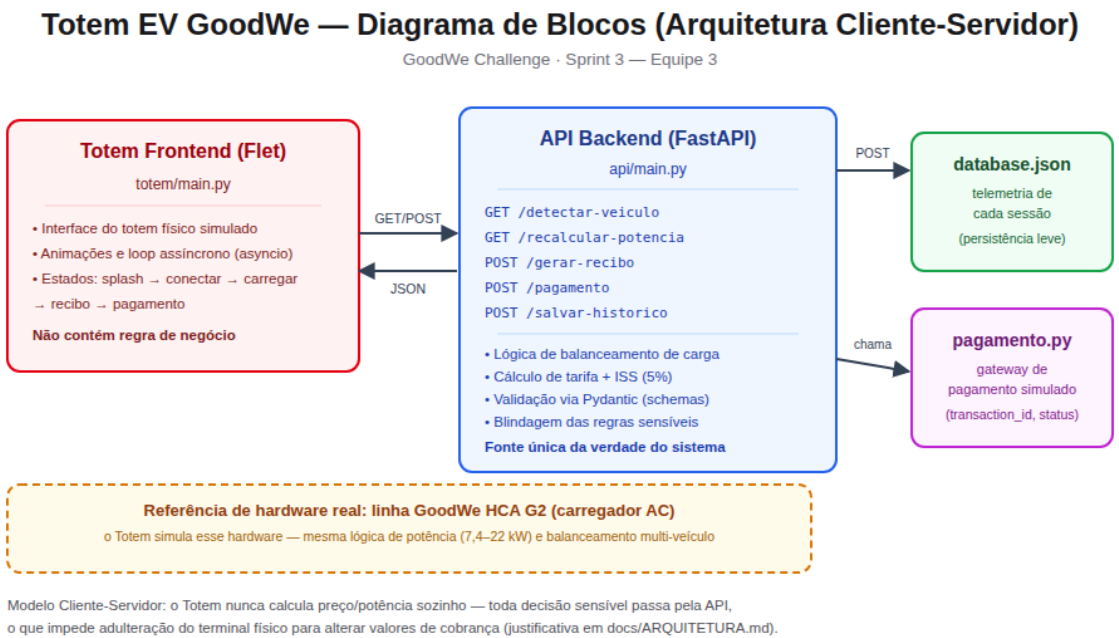
Josué Franco Braga	RM 569174
Andrei Henrique Santos	RM 569440
Heitor Maxímus Mucha	RM 571407
Enrico Marinho de Aquino	RM 569338
Fernando Lobato Rodrigues	RM 569377
Manoel da Silva Ferreira	RM 572045

Repositório: github.com/josuec3po/ev-challenge-smartgrid-grupo03

Esquema Detalhado de Integração

A arquitetura evoluiu de um sistema monolítico (interface, regras de negócio e cálculos no mesmo arquivo) para um modelo **Cliente-Servidor** desacoplado, simulando um ambiente real de IoT e infraestrutura de carregamento urbano baseado na linha GoodWe HCA G2.

Diagrama de Blocos

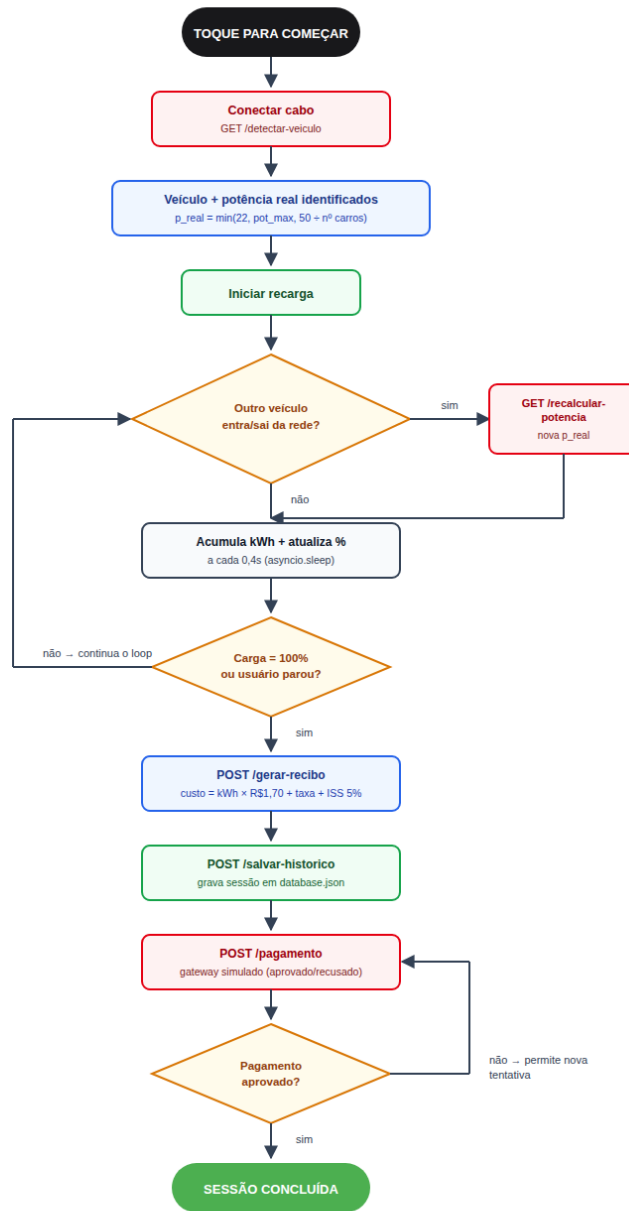


O Totem (Flet) nunca calcula preço ou potência sozinho — ele apenas exibe o que a API (FastAPI) retorna. Toda regra sensível (balanceamento de carga, tarifação, aprovação de pagamento) roda no servidor, e cada sessão concluída é persistida em database.json.

Fluxograma da Sessão de Recarga

Fluxograma — Sessão de Recarga no Totem

do toque inicial ao pagamento e persistência



Toda decisão de preço, potência e aprovação acontece no servidor (FastAPI) — o Totem só exibe o resultado, nunca calcula sozinho.

Fluxo completo de uma sessão: identificação do veículo, loop de recarga com rebalanceamento dinâmico de potência a cada entrada/saída de veículo na rede, geração de recibo, persistência do histórico e pagamento.

Passo a passo da integração

1. A interface visual no totem físico simulado (Flet) solicita uma conexão HTTP para a API (FastAPI) no servidor.
2. A API verifica e retorna o status e modelo do veículo para a interface visual.

3. O simulador inicia o loop de recarga aplicando a variação de potência com base no balanço da rede.
4. Ao final, a interface envia os dados de consumo simulado via POST para a API.
5. A API calcula a tarifa, injeta carimbo de tempo e registra o histórico no banco de dados (database.json).
6. A API retorna o recibo de validação para exibição ao usuário final.

Justificativa Técnica das Escolhas

Linguagem Python: Escolhida pela alta legibilidade e vasta biblioteca para manipulação de dados e cálculos matemáticos, essencial para a simulação física de energia.

Backend em FastAPI: Framework web moderno e assíncrono. Ideal para suportar múltiplas conexões simultâneas de totens reais de forma rápida e eficiente.

Frontend em Flet: Permitiu a criação de uma interface gráfica interativa (GUI) diretamente em Python, simulando o painel de um totem físico sem a complexidade de stacks web pesadas.

Persistência em JSON (database.json): Estrutura de dados leve e universal, perfeita para aplicações de telemetria e prototipagem que requerem gravação estruturada sem a sobrecarga de um banco de dados relacional complexo nesta fase.

Arquitetura Cliente-Servidor: Garante a escalabilidade do sistema. O totem apenas exibe e simula o hardware, enquanto regras sensíveis de cobrança, autorização e banco de dados ficam blindadas no servidor.

Resultados e Dados Funcionais Apresentados

O protótipo atinge o status funcional ponta a ponta. Através da execução do software, apresentamos os seguintes dados funcionais:

Funcionalidade	Descrição Técnica
Identificação Dinâmica	O sistema detecta automaticamente os modelos de veículos (14 modelos cadastrados, de BYD Dolphin a Porsche Taycan) e suas respectivas capacidades máximas de bateria para otimizar o ciclo de carga.
Balanceamento de Carga	A potência da recarga oscila dinamicamente (ex: de 22 kW para valores menores) em resposta à entrada simulada de outros veículos na rede local — fórmula $p_{real} = \min(22, potência_{máx_carro}, 50 \div n^o \text{ carros})$.
Cobrança Automática	Cálculo de tarifa em tempo real: consumo (kWh $\times$ R\$1,70/kWh) + taxa fixa de R\$5,00 + ISS de 5% sobre o subtotal, sem intervenção humana.
Coleta de Telemetria	Geração e armazenamento de histórico contendo ID da sessão, modelo, energia (kWh), tempo (minutos) e valor financeiro em database.json.
Gateway de Pagamento	Simulação de aprovação/recusa de pagamento com geração de transaction_id único, tratando erros de forma isolada (PaymentError).

Exemplo real de sessão (calculado pela API em execução)

Veículo	Hyundai Ioniq 5 (Top) — bateria NMC, 77,4 kWh, 11 kW AC máx.
Consumo acumulado	77,40 kWh (carga completa)
Consumo (kWh × R\$1,70)	R\$ 131,58
Taxa fixa	R\$ 5,00
ISS (5% do subtotal)	R\$ 6,83
Total cobrado	R\$ 143,41

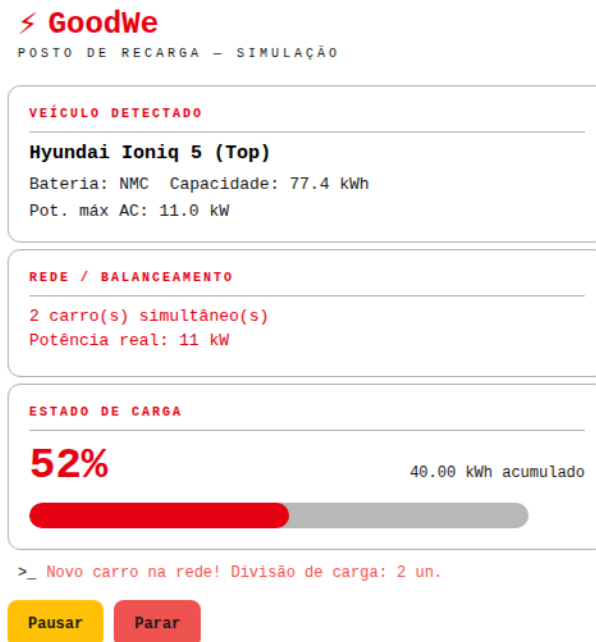
Imagens do Protótipo (Simulado)

Capturas de tela do totem em funcionamento (interface Flet), reproduzindo fielmente o layout, cores e textos definidos em totem/main.py, com dados reais retornados pela API em execução local.

1. Tela inicial — aguardando toque do usuário



2. Sessão de recarga em andamento — balanceamento dinâmico



Veículo identificado, potência real ajustada para 11 kW após entrada de um segundo veículo na rede (balanceamento automático), progresso de carga em 52% com 40,00 kWh acumulados.

3. Recibo automático ao final da sessão



POSTO DE RECARGA - SIMULAÇÃO

VEÍCULO DETECTADO

Hyundai Ioniq 5 (Top)

Bateria: NMC Capacidade: 77.4 kWh

Pot. máx AC: 11.0 kW

REDE / BALANCEAMENTO

1 carro(s) simultâneo(s)

Potência real: 11 kW

ESTADO DE CARGA

100%

77.40 kWh acumulado

>_ Bateria 100% - Finalizado

RECIBO GOODWE

Consumo (77.40 kWh): R\$ 131.58

Taxa fixa: R\$ 5.00

ISS (5%): R\$ 6.83

TOTAL: R\$ 143.41

Pagar

Ao atingir 100% de carga, o sistema gera o recibo automaticamente (consumo, taxa fixa e ISS) e libera o botão de pagamento — sem qualquer intervenção manual de um operador.

Conexão com os Conteúdos da Disciplina

(Pensamento Computacional e Automação com Python)

O projeto é uma aplicação direta dos quatro pilares do pensamento computacional sobre um problema real de automação, implementados em Python de ponta a ponta — da interface do totem ao backend que toma as decisões de negócio:

1. Decomposição: O sistema foi quebrado em módulos independentes e testáveis isoladamente: `totem/main.py` (interface e interação), `api/main.py` (regras de negócio e roteamento), `api/pagamento.py` (gateway de pagamento) e `database.json` (persistência). Cada módulo tem uma única responsabilidade — foi essa decomposição, aliás, que motivou a própria refatoração do sistema de monolítico para Cliente-Servidor (`docs/ARQUITETURA.md`), permitindo que os integrantes trabalhassem em paralelo sem conflitos de versão.

2. Abstração: A API expõe rotas simples (GET `/detectar-veiculo`, POST `/gerar-recibo`) que escondem toda a complexidade de cálculo por trás delas — o totem não sabe (nem precisa saber) como a potência real é calculada ou como a tarifa é composta, só consome o JSON de resposta. Os modelos Pydantic (`ReciboRequest`, `PagamentoRequest`, `BancoDadosRequest`) abstraem a validação de dados: o desenvolvedor declara o formato esperado e o FastAPI garante o contrato automaticamente, rejeitando requisições malformadas antes mesmo de o código de negócio ser executado.

3. Reconhecimento de padrões: O balanceamento de carga generaliza, numa única fórmula reaproveitada em dois pontos do código (`detectar_veiculo` e `recalcular_potencia`), o padrão comum a qualquer cenário de múltiplos veículos simultâneos: $p_real = \min(\text{potência_máxima_do_carregador}, \text{potência_máxima_do_carro}, 50 \div \text{número_de_carros})$. Em vez de tratar cada combinação de veículos como um caso especial, o sistema reconhece que todas seguem a mesma regra de divisão proporcional da rede.

4. Design de algoritmos e automação com Python: Toda a operação do totem é automatizada sem intervenção humana, usando ferramentas típicas de automação em Python: corrotinas assíncronas (`async def`, `await asyncio.sleep(0.4)`) mantêm o loop de carregamento e a interface responsivos ao mesmo tempo, sem travar a tela; a biblioteca `requests` automatiza a comunicação HTTP entre totem e API a cada evento (conectar, recalcular potência, gerar recibo, salvar histórico); decoradores (`@app.get`, `@app.post`) automatizam o roteamento de cada função Python como um endpoint de rede; e estruturas condicionais decidem sozinhas quando rebalancear potência, quando liberar o botão de pagamento e quando persistir a sessão em `database.json` — o algoritmo caminha do início ao fim da recarga sem que nenhum operador precise clicar em nada além do toque inicial.

Instruções de Funcionamento (Como Executar)

Para validar o funcionamento do sistema em ambiente local, siga os passos abaixo:

1. Clone este repositório em sua máquina local.
2. Certifique-se de ter o Python instalado (versão 3.10 ou superior).
3. Instale as dependências: `pip install fastapi uvicorn flet requests pydantic`
4. Inicie o Servidor: na raiz do projeto, rode `uvicorn api.main:app --reload`
5. Inicie o Totem: em outro terminal, rode `python totem/main.py`
6. Interaja com a interface para realizar uma recarga. Ao finalizar, verifique o arquivo `database.json` atualizado automaticamente com os dados da telemetria.